Java Basic and Syntax

Lecture 3

Instructor:
Shovon Mandal (SM), Lecturer
Department of Computer Science and Engineering (CSE),
Northern University & Business Technology Khulna (NUBTK)

Java Virtual Machine (JVM)

- The JVM (Java Virtual Machine) is a virtual machine, an abstract computer that has its own memory, stack, heap, etc. It runs on the host OS and places its demands for resources on it.

Note:

JDK = Java Runtime Environment (JRE) + Development tools

JRE = Java Virtual Machine (JVM) + Libraries to run the application

JVM = Only Runtime environment for executing the Java byte code.

Basic Terms in Java

- **Object** – Objects have states and behaviors. Example: A dog has states - color, name, breed as well as behavior such as wagging their tail, barking, eating. An object is an instance of a class.
- **Class** – A class can be defined as a template/blueprint that describes the behavior/state that the object of its type supports.
- **Methods** – A method is basically a behavior. A class can contain many methods. It is in methods where the logics are written, data is manipulated, and all the actions are executed.
- **Instance Variables** – Each object has its unique set of instance variables. An object's state is created by the values assigned to these instance variables.

Basic Syntax

- **Case Sensitivity** - Java is case sensitive, which means identifier Hello and hello would have different meaning in Java.
- **Class Names** - For all class names the first letter should be in Upper Case. If several words are used to form a name of the class, each inner word's first letter should be in Upper Case.

  Example - class MyFirstJavaClass
- **Method Names** - All method names should start with a Lower Case letter. If several words are used to form the name of the method, then each inner word's first letter should be in Upper Case.

  Example - public void myMethodName()
- **Program File Name** - Name of the program file should exactly match the class name. When saving the file, you should save it using the class name (Remember Java is case sensitive) and append '.java' to the end of the name (if the file name and the class name do not match, your program will not compile).

  Example - Assume 'MyFirstJavaProgram' is the class name. Then the file should be saved as 'MyFirstJavaProgram.java'
- **public static void main(String args[])** - Java program processing starts from the main() method which is a mandatory part of every Java program.

Java Identifiers

- All identifiers should begin with a letter (A to Z or a to z), currency character ($) or an underscore (_).
- After the first character, identifiers can have any combination of characters.
- A key word cannot be used as an identifier.
- Most importantly, identifiers are case sensitive.
- Examples of legal identifiers: age, $salary, _value, __1_value.
- Examples of illegal identifiers: 123abc, -salary.

Java Modifiers

- **Access Modifiers** – default, public , protected, private

- **Non-access Modifiers** – final, abstract etc

White Space

- Spaces, blank lines, and tabs are collectively called *white space*
- White space is used to separate words and symbols in a program
- Extra white space is ignored
- Example:

```
public              class
      HelloWorld     { public static void


          main(String[]
args)                                    {
    System.out.println("Hello World!")
                    ;}}
```

- Valid, but bad Indentation

Identifiers: Semantics

- Identifiers names can come from the following sources
  - Fixed in Java as *reserved words*
    - public, class, static, void, int, if, else, for, while, switch, case, break, try, catch, return, new, this, extends, implements, import
  - Chosen by the programmer to denote something
    - HelloWorld, main, args
  - Chosen by a programmer whose code we use:
    - String, System, out, println

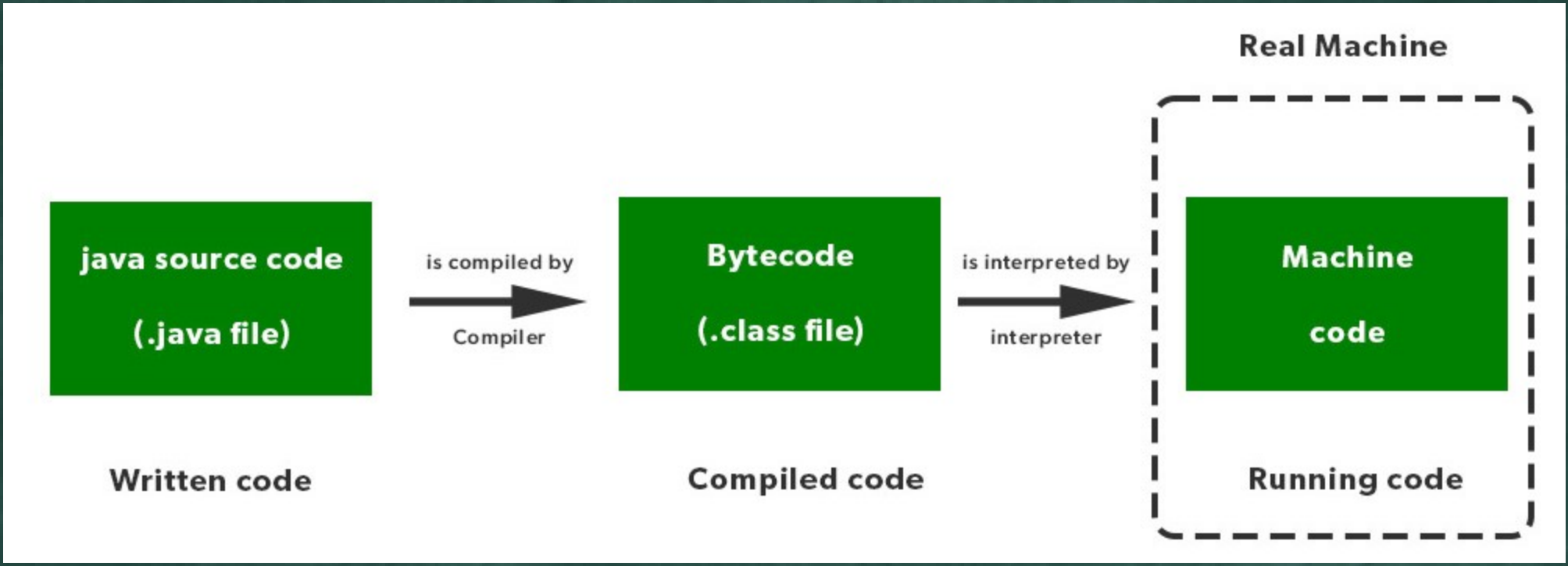
Separators

Symbol	Name	Purpose
( )	Parentheses	Used to contain lists of parameters in method definition and invocation. Also used for defining precedence in expressions, containing expressions in control statements, and surrounding cast types.
{ }	Braces	Used to contain the values of automatically initialized arrays. Also used to define a block of code, for classes, methods, and local scopes.
[ ]	Brackets	Used to declare array types. Also used when dereferencing array values.
;	Semicolon	Terminates statements.
,	Comma	Separates consecutive identifiers in a variable declaration. Also used to chain statements together inside a **for** statement.
.	Period	Used to separate package names from subpackages and classes. Also used to separate a variable or method from a reference variable.
::	Colons	Used to create a method or constructor reference.
...	Ellipsis	Indicates a variable-arity parameter.
@	Ampersand	Begins an annotation.

Java Keywords

abstract	assert	boolean	break	byte	case
catch	char	class	const	continue	default
do	double	else	enum	exports	extends
final	finally	float	for	goto	if
implements	import	instanceof	int	interface	long
module	native	new	open	opens	package
private	protected	provides	public	requires	return
short	static	strictfp	super	switch	synchronized
this	throw	throws	to	transient	transitive
try	uses	void	volatile	while	with
_					

Roles of Java Compiler and Interpreter



Compiler VS Interpreter

- The interpreter scans the program line by line and translates it into machine code whereas the compiler scans the entire program first and then translates it into machine code.
- The interpreter shows one error at a time whereas the compiler shows all errors and warnings at the same time.
- In the interpreter, the error occurs after scanning each line whereas, in the compiler, the error occurs after scanning the whole program.
- In an interpreter, debugging is faster whereas, in the compiler, debugging is slow.
- Execution time is more in the interpreter whereas execution time is less in the compiler.
- An interpreter is used by languages such as Java, and Python, and the compiler is used by languages such as C, C++, etc.

Thank You